30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 6

Jetpack Essentials — ViewModel, SavedStateHandle, StateFlow, Room, DataStore & Paging



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 6

Why Jetpack internals matter in 2026

"I've used ViewModel and Room" is no longer enough. Interviewers probe **how they work inside** and **what they don't do**:

- ViewModel vs `SavedStateHandle`: what survives rotation, and what survives process death
- LiveData isn't deprecated — but `StateFlow` is the default for new and multiplatform code
- Room relations, migrations and transactions: where production data gets lost
- DataStore replaced SharedPreferences; Paging 3 handles large lists
- ViewModel, Room, DataStore and the Paging core now run in KMP common code

Level	Questions
Junior	ViewModel internals · LiveData vs StateFlow · Room basics
Mid	SavedStateHandle · Factories & CreationExtras · Relations · DataStore
Senior	Migrations · Room under load · Paging 3
Architect	Room in KMP · Production migrations · One database or many



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: what survives what

	Rotation	Process death	Swiped away
`remember { }`	no	no	no
`rememberSaveable`	yes	yes	no
`ViewModel`	yes	no	no
`SavedStateHandle`	yes	yes	no
`Room · DataStore`	yes	yes	yes

- **ViewModel —** an in-memory cache for one screen
- **SavedStateHandle —** small keys that survive system-initiated process death
- **Room / DataStore —** the truth on disk; after process death the screen is rebuilt from *keys* + *disk*

THE QUESTION BEHIND EVERY JETPACK ANSWER

What must survive what — and who owns it? Pick the cheapest holder that survives what you need.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 6

→

Junior

ViewModel internals · LiveData vs StateFlow · Room basics



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. How does a ViewModel survive rotation?

- `by viewModels()` / `viewModel()` asks `ViewModelProvider` for the instance stored in the owner's **`ViewModelStore`** — a map keyed by class name. It's created only if missing.
- On rotation, `ComponentActivity` passes its store to the new instance as a **non-configuration instance** → the same ViewModel object comes back.
- When the activity finishes for good, the store is cleared.

Rotation	finish() / Back
old Activity destroyed, store handed over new Activity: same store, same ViewModel instance	ON_DESTROY and not isChangingConfigurations → viewModelStore.clear() → onCleared(), scope cancelled

FOLLOW-UP

"Where do a Fragment's ViewModels live?" — In a `FragmentManagerViewModel`, which is itself a ViewModel in the activity's store. That's why they survive rotation too.



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Owners and cleanup

Owner	Its ViewModels are cleared when
`ComponentActivity`	It finishes — not on rotation
`Fragment`	It's removed for good — not while on the back stack
`NavBackStackEntry`	The destination is popped
Compose `viewModel()`	Uses the nearest `LocalViewModelStoreOwner`

```
class PlayerViewModel(player: AudioPlayer) : ViewModel() {
    init {
        addCloseable(player) // closed when the VM is cleared
    }
}
```

INTERVIEW TRAP

"ViewModel is lifecycle-aware." (last year's version said so) — It's lifecycle-**scoped**: it never observes a lifecycle. It simply lives in a `ViewModelStore` until the owner clears it.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. LiveData vs StateFlow

	LiveData	StateFlow
Lifecycle	Built in: delivers only to STARTED/RESUMED observers	None — collect with `repeatOnLifecycle` / `collectAsStateWithLifecycle`
Initial value	Optional (can be unset)	Required
Same value again	`setValue` always notifies	Skipped (compared with `equals`)
Threads	`setValue` main only; `postValue` from any thread	`value` / `update {}` from any thread
Operators	`map`, `switchMap`, `distinctUntilChanged`, `MediatorLiveData`	All Flow operators
Platform	Android only	Kotlin — KMP, plain JVM tests

INTERVIEW TRAP

"LiveData has no `distinctUntilChanged()`, you need a custom transformer." (v1) — Wrong: `distinctUntilChanged()`, `map` and `switchMap` ship with `lifecycle-livedata`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Collecting StateFlow safely

```
// Views: collect only while the view is at least STARTED
val owner = viewLifecycleOwner
owner.lifecycleScope.launch {
    owner.repeatOnLifecycle(Lifecycle.State.STARTED) {
        viewModel.uiState.collect(::render)
    }
}

// Compose
val state by viewModel.uiState.collectAsStateWithLifecycle()

// Bridges while migrating
val flow: Flow<User> = userLiveData.asFlow()
val live: LiveData<User> = userFlow.asLiveData()
```

INTERVIEW TRAP

"`postValue` *delivers every value."* — No. Several posts before the main thread runs → only the **last** one is dispatched. Fine for state, wrong for events.

INTERVIEW TRAP

"LiveData is deprecated." — It isn't. It's stable and supported; StateFlow is simply the better default for new, Compose and KMP code. `launchWhenStarted` **is** deprecated — use `repeatOnLifecycle`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Room basics: entity, DAO, database

```
@Entity(tableName = "notes")
data class NoteEntity(
    @PrimaryKey val id: String,
    val title: String,
    val updatedAt: Long,
)

@Dao
interface NoteDao {
    @Query("SELECT * FROM notes ORDER BY updatedAt DESC")
    fun observeAll(): Flow<List<NoteEntity>>

    @Upsert suspend fun upsert(note: NoteEntity)
}

@Database(entities = [NoteEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun noteDao(): NoteDao
}
```

SQL is checked **at compile time** (Room's compiler runs on KSP). A `Flow` query re-emits whenever its tables change.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Rules interviewers check

- `suspend` DAO functions are **main-safe**: Room runs them on its own executors.
- A query on the main thread throws `IllegalStateException` — unless you opt out, which you shouldn't.
- One `RoomDatabase` instance per process — let DI own it (Day 7).
- Built-in column types: primitives, `String`, `ByteArray`, enums. Everything else needs a `@TypeConverter`.

INTERVIEW TRAP

Wrapping DAO calls in `withContext(Dispatchers.IO)`. Redundant — the `suspend` DAO is already main-safe; you only add a thread hop.

INTERVIEW TRAP

`exportSchema = false` plus a hand-written double-checked-locking singleton (v1). Keep schema export on (it's the default) with the Room Gradle plugin — `room { schemaDirectory("$projectDir/schemas") }` — so migrations can be generated and tested. Commit the JSON files.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 6



Mid-Level

SavedStateHandle · Factories & CreationExtras · Relations · DataStore



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. SavedStateHandle: how it works

```
class SearchViewModel(
    private val handle: SavedStateHandle,
    private val repo: SearchRepository,
) : ViewModel() {
    // Restored after system-initiated process death
    val query: StateFlow<String> =
        handle.getStateFlow(QUERY, "")

    val results: StateFlow<List<Repo>> = query
        .flatMapLatest { repo.search(it) }
        .stateIn(
            viewModelScope,
            SharingStarted.WhileSubscribed(5_000),
            emptyList(),
        )

    fun onQueryChange(q: String) { handle[QUERY] = q }
}
```

- Created from `CreationExtras`: the owner's `SavedStateRegistry` plus **default args** (intent extras, Fragment arguments, navigation route)
- Registers a `SavedStateProvider`: when the owner saves state, the handle's map is written into the owner's `Bundle`



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Limits you must know

- Values must fit in a `Bundle`: primitives, `String`, `Parcelable`… Save **ids, a query, a selected tab** — not lists of data.
- The Bundle travels through Binder. The process's transaction buffer is 1 MB, shared by everything in flight → too much state = `TransactionTooLargeException`.
- It's gone when the user swipes the task away or force-stops the app — that's a fresh start.

INTERVIEW TRAP

"SavedStateHandle writes to disk." — No. It's the instance-state `Bundle`, kept by the system. For data that must survive everything, use Room or DataStore.

INTERVIEW TRAP

Putting the whole `UiState` into `SavedStateHandle` "to be safe". You risk `TransactionTooLargeException` and restore stale data. Save keys; reload data from the repository (Day 3).

FOLLOW-UP

"*`rememberSaveable` or `SavedStateHandle`?*" — `rememberSaveable` for UI element state (expanded, scroll); `SavedStateHandle` for state the screen logic needs.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Creating ViewModels: factories & CreationExtras

```
class DetailViewModel(
    private val repo: DetailRepository,
    handle: SavedStateHandle,
) : ViewModel() {
    companion object {
        val Factory = viewModelFactory {
            initializer {
                val app = this[APPLICATION_KEY] as MyApp
                DetailViewModel(
                    repo = app.container.detailRepository,
                    handle = createSavedStateHandle(),
                )
            }
        }
    }
}
// Fragment: by viewModels { DetailViewModel.Factory }
// Compose:  viewModel(factory = DetailViewModel.Factory)
```

`CreationExtras` carry what the factory needs: `APPLICATION_KEY`, `VIEW_MODEL_STORE_OWNER_KEY`, `SAVED_STATE_REGISTRY_OWNER_KEY`, `DEFAULT_ARGS_KEY`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. What happens inside `get()`

1. `ViewModelProvider(owner, factory)[DetailViewModel::class]` builds a key from the class name (or your `key`).
2. Found in the `ViewModelStore` → returned; **the factory is not called**.
3. Missing → `factory.create(modelClass, extras)` → stored under that key.

Hilt's `@HiltViewModel` and Koin's `viewModelOf` generate this factory for you (Day 7).

INTERVIEW TRAP

`val vm = DetailViewModel(repo)` in `onCreate()`. It isn't in any store: a new instance on every rotation, `onCleared()` never runs, and `viewModelScope` is never cancelled.

INTERVIEW TRAP

Passing a new `itemId` to a factory after rotation. The factory ran once; you get the old instance back. Per-screen arguments belong in `SavedStateHandle` (navigation args) — or use a distinct `key`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Room relations: one-to-many & many-to-many

```
data class UserWithPlaylists(
    @Embedded val user: User,
    @Relation(parentColumn = "userId", entityColumn = "ownerId")
    val playlists: List<Playlist>,
)

@Entity(primaryKeys = ["playlistId", "songId"])
data class PlaylistSongCrossRef(
    val playlistId: Long,
    val songId: Long,
)

data class PlaylistWithSongs(
    @Embedded val playlist: Playlist,
    @Relation(
        parentColumn = "playlistId",
        entityColumn = "songId",
        associateBy = Junction(PlaylistSongCrossRef::class),
    )
    val songs: List<Song>,
)
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Querying relations correctly

```
@Dao
interface MusicDao {
    @Transaction
    @Query("SELECT * FROM User")
    fun usersWithPlaylists(): Flow<List<UserWithPlaylists>>

    // Alternative: one JOIN returning a multimap
    @Query(
        "SELECT * FROM User " +
            "JOIN Playlist ON User.userId = Playlist.ownerId"
    )
    suspend fun playlistsByUser(): Map<User, List<Playlist>>
}
```

- Room runs **one query for the parents and more for the children**. `@Transaction` makes them one consistent read — Room warns if it's missing.

INTERVIEW TRAP

"*`@Relation` supports only one level of nesting.*" (v1) — Wrong. Nest by pointing `@Relation(entity = Playlist::class, …)` at a class that has its own relation, e.g. `List<PlaylistWithSongs>`.

INTERVIEW TRAP

`@Relation` is **not** a foreign key. For integrity add `ForeignKey(onDelete = CASCADE)` on the child — and index that column (Room warns otherwise).



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. DataStore vs SharedPreferences vs Room

	SharedPreferences	DataStore	Room
API	Sync reads; `apply()` / `commit()`	`Flow` + `suspend` writes	DAO + SQL
Main-thread risk	Yes: first load, `commit()`, pending `apply()` flushed at `onStop`	No	No
Errors	Silent	`IOException` in the Flow	Exceptions
Consistency	No transactions	Atomic read-modify-write	Transactions
Use for	Legacy only	Small settings, flags	Relational or large data

- **Preferences DataStore:** typed keys, no schema
- **Typed DataStore** (Proto, or JSON via kotlinx.serialization): a full data class with a `Serializer`
- Not a database: no queries, and every edit rewrites the whole file
- DataStore is multiplatform — the same code in `commonMain`



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. DataStore in code

```
val Context.settings by preferencesDataStore(
    name = "settings",
    produceMigrations = { ctx ->
        listOf(SharedPreferencesMigration(ctx, "prefs"))
    },
)
private val DARK_MODE = booleanPreferencesKey("dark_mode")

class SettingsRepository(
    private val store: DataStore<Preferences>,
) {
    val darkMode: Flow<Boolean> = store.data
        .catch { e ->
            if (e !is IOException) throw e
            emit(emptyPreferences())
        }
        .map { it[DARK_MODE] ?: false }

    suspend fun setDarkMode(on: Boolean) {
        store.edit { it[DARK_MODE] = on }
    }
}
```

INTERVIEW TRAP

Two DataStore instances for the **same file** in one process → `IllegalStateException`. Declare it once: a top-level delegate or a DI singleton.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 6



Senior

Migrations · Room under load · Paging 3



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Migrations: automatic first

```
@Database(
    entities = [User::class],
    version = 3,
    autoMigrations = [
        AutoMigration(from = 1, to = 2),
        AutoMigration(from = 2, to = 3, spec = AppDb.V3::class),
    ],
)
abstract class AppDb : RoomDatabase() {
    @RenameColumn(
        tableName = "User",
        fromColumnName = "name",
        toColumnName = "fullName",
    )
    class V3 : AutoMigrationSpec
}
```

- AutoMigration diffs the **exported schema JSON** of two versions — no export, no auto-migration.
- Ambiguous changes (rename, delete) need a spec, or the build fails. Everything else: a manual `Migration`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Manual migrations — and testing them

```
val MIGRATION_3_4 = object : Migration(3, 4) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL("ALTER TABLE User ADD COLUMN email TEXT")
    }
}

@get:Rule val helper = MigrationTestHelper(
    InstrumentationRegistry.getInstrumentation(),
    AppDb::class.java,
)

@Test fun migrate3To4() {
    helper.createDatabase(DB_NAME, 3).close()
    helper.runMigrationsAndValidate(
        DB_NAME, 4, true, MIGRATION_3_4,
    )
}
```

INTERVIEW TRAP

"Without a migration, Room deletes and recreates the database." (v1) — No: Room throws `IllegalStateException` on open. Data is wiped only if you opted into `fallbackToDestructiveMigration…`.

INTERVIEW TRAP

"You must also write 1→3 next to 1→2 and 2→3." (v1) — Room chains migrations. A direct 1→3 is an optional shortcut.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Room under load: indexes & transactions

```
@Entity(
    indices = [
        Index("ownerId"),
        Index(value = ["email"], unique = true),
    ],
)
data class Account(
    @PrimaryKey val id: Long,
    val ownerId: Long,
    val email: String,
)

// One transaction: one commit instead of thousands
suspend fun replaceAll(items: List<ItemEntity>) =
    db.withTransaction {
        itemDao.deleteAll()
        itemDao.insertAll(items)
    }
```

- Index the columns used in `WHERE`, `JOIN`, `ORDER BY` and foreign keys. Every index costs space and slows writes.
- Batch writes in one transaction: much faster, and readers never see half the batch.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Observable queries & locking

- A `Flow` query re-runs when **any** table it reads is written — even rows outside its result. Room emits the new result even if it's equal → add `distinctUntilChanged()`.
- Don't observe a huge `SELECT *`: select only needed columns, or page it (Q10).
- Room enables **write-ahead logging (WAL)** on most devices: readers don't block the writer — but there is still only **one writer** at a time. Keep transactions short.

INTERVIEW TRAP

"A partial update locks only the changed columns." (v1) — SQLite has no column or row locks: a write locks the **whole database**. Partial updates (`@Update(entity = Account::class)` with a partial class, or `UPDATE … SET`) save serialization and I/O, not lock contention.

MEASURE IT

Android Studio's **Database Inspector** runs live queries on a device; run `EXPLAIN QUERY PLAN` there to confirm a query uses your index.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Paging 3: the pieces

```
PagingSource ─▶ Pager(PagingConfig) ─▶ Flow<PagingData>
(loads pages)   (in the ViewModel)     .cachedIn(viewModelScope)
                                                 │
            LazyColumn ◀── collectAsLazyPagingItems()
```

```kotlin
// DAO — Room generates the PagingSource (room-paging)
@Query("SELECT * FROM articles ORDER BY publishedAt DESC")
fun pagingSource(): PagingSource<Int, ArticleEntity>

// ViewModel
val articles: Flow<PagingData<Article>> =
    Pager(PagingConfig(pageSize = 30)) { dao.pagingSource() }
        .flow
        .map { page -> page.map { it.toDomain() } }
        .cachedIn(viewModelScope)
```

When the table changes, Room **invalidates** the `PagingSource`; the `Pager` creates a new one and reloads around the current position.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Paging in the UI

```
val items = viewModel.articles.collectAsLazyPagingItems()

LazyColumn {
    items(
        count = items.itemCount,
        key = items.itemKey { it.id },
    ) { index ->
        val article = items[index]
        if (article != null) ArticleRow(article)
        else ArticlePlaceholder()
    }
}
// Errors: loadState.refresh is LoadState.Error → items.retry()
```

INTERVIEW TRAP

Forgetting `cachedIn(viewModelScope)`. Rotation reloads from scratch, and collecting the same `PagingData` twice throws. Transform (`map`, `filter`) **before** `cachedIn`.

INTERVIEW TRAP

`DataSource.Factory` / `PagedList` (v1) is the Paging 2 API — deprecated. Paging 3 uses `PagingSource` and `PagingData`.

FOLLOW-UP

"Network + database?" — Room is the source, a `RemoteMediator` fills it (Day 14).



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 6



Architect

Room in KMP · Production migrations · One database or many



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Room in Kotlin Multiplatform

```kotlin
// commonMain
@Database(entities = [NoteEntity::class], version = 1)
@ConstructedBy(AppDatabaseConstructor::class)
abstract class AppDatabase : RoomDatabase() {
    abstract fun noteDao(): NoteDao
}

// Room's compiler generates the actual objects
@Suppress("KotlinNoActualForExpect")
expect object AppDatabaseConstructor :
    RoomDatabaseConstructor<AppDatabase> {
    override fun initialize(): AppDatabase
}

fun buildDatabase(b: RoomDatabase.Builder<AppDatabase>) = b
    .setDriver(BundledSQLiteDriver())
    .setQueryCoroutineContext(Dispatchers.IO)
    .build()
```

Each platform only creates the builder: `Room.databaseBuilder<AppDatabase>(context, path)` on Android, `Room.databaseBuilder<AppDatabase>(name = path)` on iOS.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. What changes in KMP

- DAOs in common code must be `suspend` or return `Flow` — no blocking DAOs, no `LiveData`
- Migrations and callbacks use `SQLiteConnection`, not the Android-only `SupportSQLiteDatabase`
- **Bundled SQLite:** the same modern SQLite everywhere, at the cost of a native library per ABI

Library	In `commonMain`?
ViewModel, `viewModelScope`	Yes — Lifecycle 2.8+
Room	Yes — 2.7+
DataStore	Yes
Paging	Core (`paging-common`)
LiveData, WorkManager	No — Android only

INTERVIEW TRAP

"Just move the Android Room code to `commonMain`." — Blocking DAOs, `LiveData`/Rx return types, `SupportSQLite*` in migrations and converters that use Android types all have to go first.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Production migrations: a strategy

Change	Strategy
New table, nullable or defaulted column	`AutoMigration`
Rename or delete a column or table	`AutoMigration` + spec
Type change, split/merge columns, data fixes	Manual: create new table → copy + transform → drop → rename
Cache-only database	Destructive fallback is acceptable
User installs an older version	Throws by default — decide explicitly (`fallbackToDestructiveMigrationOnDowngrade`)

- **Never edit a migration that has shipped** — add a new version.
- In CI, test from **every released schema version** to the current one, with realistic data.
- Each migration runs in a transaction, then Room **validates** the schema. A mismatch crashes on open instead of silently corrupting data.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. A migration that transforms data

```
val MIGRATION_7_8 = object : Migration(7, 8) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL(
            "CREATE TABLE Product_new (" +
                "id INTEGER NOT NULL, name TEXT NOT NULL, " +
                "priceCents INTEGER NOT NULL, PRIMARY KEY(id))"
        )
        db.execSQL(
            "INSERT INTO Product_new (id, name, priceCents) " +
                "SELECT id, name, " +
                "CAST(ROUND(price * 100) AS INTEGER) " +
                "FROM Product"
        )
        db.execSQL("DROP TABLE Product")
        db.execSQL("ALTER TABLE Product_new RENAME TO Product")
        db.execSQL(
            "CREATE INDEX index_Product_name ON Product(name)"
        )
    }
}
```

INTERVIEW TRAP

Forgetting to recreate indexes after rebuilding a table → Room's schema validation fails. Copy the exact `createSql` from the exported schema JSON.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. One database or many?

One database	Several databases
Transactions and JOINs across all tables	No transaction or JOIN across files
One invalidation tracker, one writer	Each file has its own connections and writer
`@Database` lists every entity → one module sees all features' tables	Each feature owns its schema and migrations
One version history	One version history per database

Default: one database in a `:core:database` module; features see repositories, not the database. **Split** when the lifecycle differs:

- A **per-user** database deleted at logout
- A **cache** database that may be dropped
- A module owned by another team or delivered separately



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. A per-user database

```
fun openUserDb(context: Context, userId: String): UserDb =
    Room.databaseBuilder(
        context, UserDb::class.java, "user-$userId.db",
    ).build()

fun deleteUserData(context: Context, db: UserDb, id: String) {
    db.close()
    context.deleteDatabase("user-$id.db")
}
```

INTERVIEW TRAP

"Separate databases add security." (v1) — Every database file sits in the same app sandbox with the same permissions. Separation is not a security boundary; encryption at rest is (Day 14, Day 19).

SENIOR ANSWER IN ONE LINE

"Memory for the screen, a Bundle for the keys, disk for the truth — and every schema change is a migration I've tested before a user runs it."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 6 · ARCHITECT

Day 6 — cheat sheet

- ViewModels live in a `ViewModelStore`: rotation hands it over, `finish()` clears it. Lifecycle-**scoped**, not lifecycle-aware.
- `SavedStateHandle` = small keys in the saved-state Bundle → survives process death, not "swipe away".
- Factories run once per owner + key; use `CreationExtras` or DI — never `MyViewModel(repo)` by hand.
- LiveData isn't deprecated; `StateFlow` + `collectAsStateWithLifecycle` is the default for new and KMP code.
- Room: KSP, `suspend` / `Flow` DAOs, no `withContext(IO)` wrappers, exported schemas in git.
- Relations: `@Relation` + `@Transaction`; nesting works; a relation is not a foreign key.
- Missing migration → crash, not wipe. AutoMigration for simple changes, SQL for transforms, tests from every shipped version.
- `Flow` queries re-run on any write to their tables → `distinctUntilChanged()`; batch writes in one transaction.
- DataStore for small settings (one instance per file); Paging 3 with `cachedIn(viewModelScope)` for big lists.
- ViewModel, Room, DataStore and the Paging core all run in KMP common code.

Day 7 is next → Dependency Injection. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700